

자동차관리법 RAG 챗봇

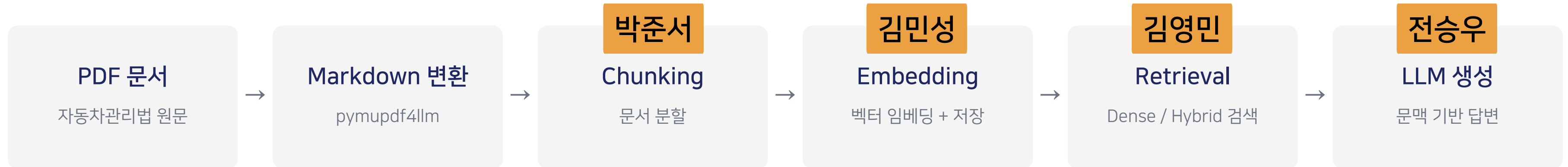
박준서 | 김민성 | 김영민 | 전승우

I 왜 이 프로젝트를 하는가?

문제 의식

- ✔ 자동차관리법과 같은 전문 법령은 조문이 방대하고 표현이 엄격해, 일반 LLM에 그대로 질문하면 사실과 다른 답변이 나오기 쉽습니다.
- ✔ 사용자는 조문 번호나 정확한 문구를 몰라도, 자신의 상황에 맞는 답을 신뢰할 수 있게 얻고 싶어 합니다.
- ✔ 따라서 문서에 근거해서만 답하는 RAG 구조로, 법령 원문을 검색해 답변을 생성하는 챗봇을 설계합니다.

I 프로젝트 개요 — 전체 파이프라인



평가 방식

- 질문 10개(EVAL_DATA)에 대해 챗봇이 생성한 답변과 정답(ground truth)을 비교
- klue/bert-base 기반 BERTScore(F1)로 의미적 유사도를 정량 측정하고, 응답 시간도 함께 기록
- LLM · Embedding · Chunking · Retrieval 각 축을 바꿔가며 결과를 비교해 최적 조합을 탐색

왜 Greedy Search인가

각 축의 개별 승자를 단순히 합쳤을 때는 축 간 상호작용으로 F1이 오히려 하락했습니다. 이에 이미 검증된 최고 조합을 출발점으로, 한 번에 변수 하나씩만 바꾸며(coordinate ascent) 재검증하는 방식을 채택했습니다.

I PDF 로더 대신 마크다운 변환을 선택한 이유

PDF 로더 직접 사용

- 페이지 단위로 텍스트가 끊김

제0조 내용이 페이지 경계에서 둘로 분리되어 문맥·검색 정확도 손실

- 레이아웃 노이즈 혼입

머리말·페이지 번호·각주가 본문 중간에 섞여 순수 조문 텍스트 파악이 어려움

- 조문 경계 탐지 불가

정규식으로 '제0조(...)' 패턴을 안정적으로 찾기 어려워 article 청킹 전략 구현이 힘들

PDF → 마크다운 변환

- 문서 전체를 하나의 연속 텍스트로 확보

`pymupdf4llm.to_markdown()`으로 페이지 구분 없이 전체를 이어 붙여 조문이 잘리지 않음

- 조문 경계를 정규식으로 안정적으로 탐지

`ARTICLE_SPLIT_PATTERN(제0조(...))`으로 article 청킹 전략을 그대로 구현 가능

- 변환 결과 캐싱으로 재현성·속도 확보

`MD_CACHE_PATH`에 저장해 재변환 없이 4가지 전략이 동일 텍스트에서 출발 — 변인 통제

I Chunking Tuning 개요

목적

법령 문서에서 조문(제0조) 단위가 유지되도록 Chunking 방식을 개선하여 Retrieval 정확도와 RAG 성능을 향상시키는 것

왜 Chunking을 변경했는가?

- Recursive 방식은 조문이 중간에서 잘려 의미 손실 발생
- Article Splitter는 조문 단위 유지로 맥락 보존
- Retriever의 검색 정확도 및 최종 답변 품질 향상

전체 파이프라인 속 위치

청킹

임베딩

검색

LLM

챗봇

I 실험 설계

공통 고정 조건

LLM
Qwen2.5-7B-Instruct (4bit, 로컬)

임베딩 모델
jhgan/ko-sroberta-multitask

검색 방식
similarity, top-k = 3

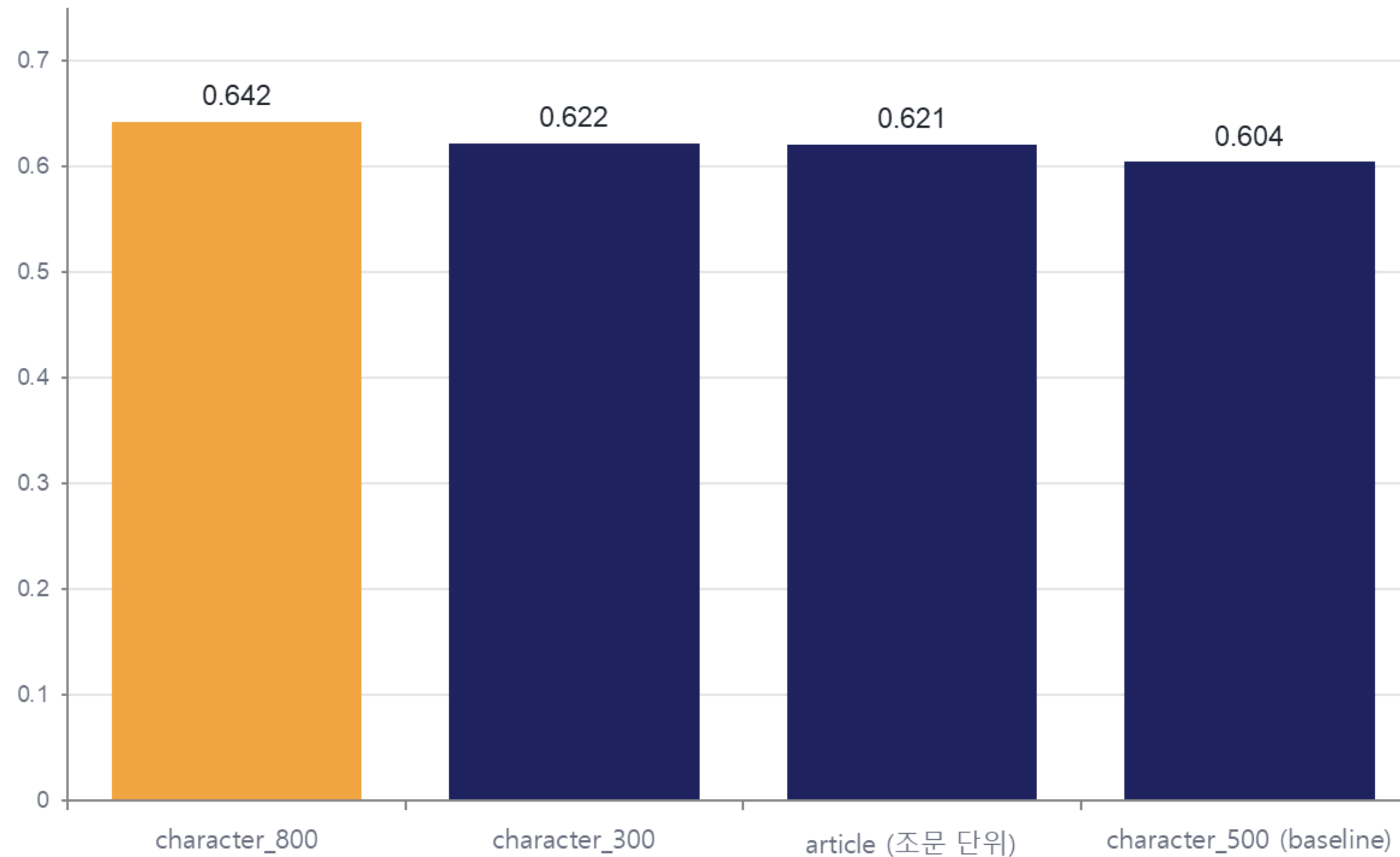
평가 데이터
자동차관리법 질의 10문항

평가 지표
BERTScore F1 등

비교 대상 청킹 전략

전략	chunk_size / overlap	분할 기준
character_300	300 / 50	문자 수 (짧게)
character_500	500 / 50 (baseline)	문자 수 (기존)
character_800	800 / 100	문자 수 (길게)
article	가변 (초과 시 800/100)	조문(제0조) 단위

I 결과 ① 정확도 (BERTScore F1)



최고 성능

0.642

character_800

baseline(0.604) 대비 + 6.3p%

순위

1. character_800 0.642

2. character_300 0.622

3. article (조문 단위) 0.621

4. character_500 (baseline) 0.604

I 다른 전략들은 왜 낮았는가

character_300 *F1 0.622* · 청크 최다(920개)

청크가 300자로 짧아 조문이 중간에서 자주 잘림 — 문맥이 끊긴 채 검색되어 답변 품질이 character_800에 못 미침. 응답은 빠르지만(10.5초) 청크 수가 3배 가까이 늘어 저장 및 검색 비용은 가장 큼.

character_500 (baseline) *F1 0.604(최저)* · 응답 최장(13.4초)

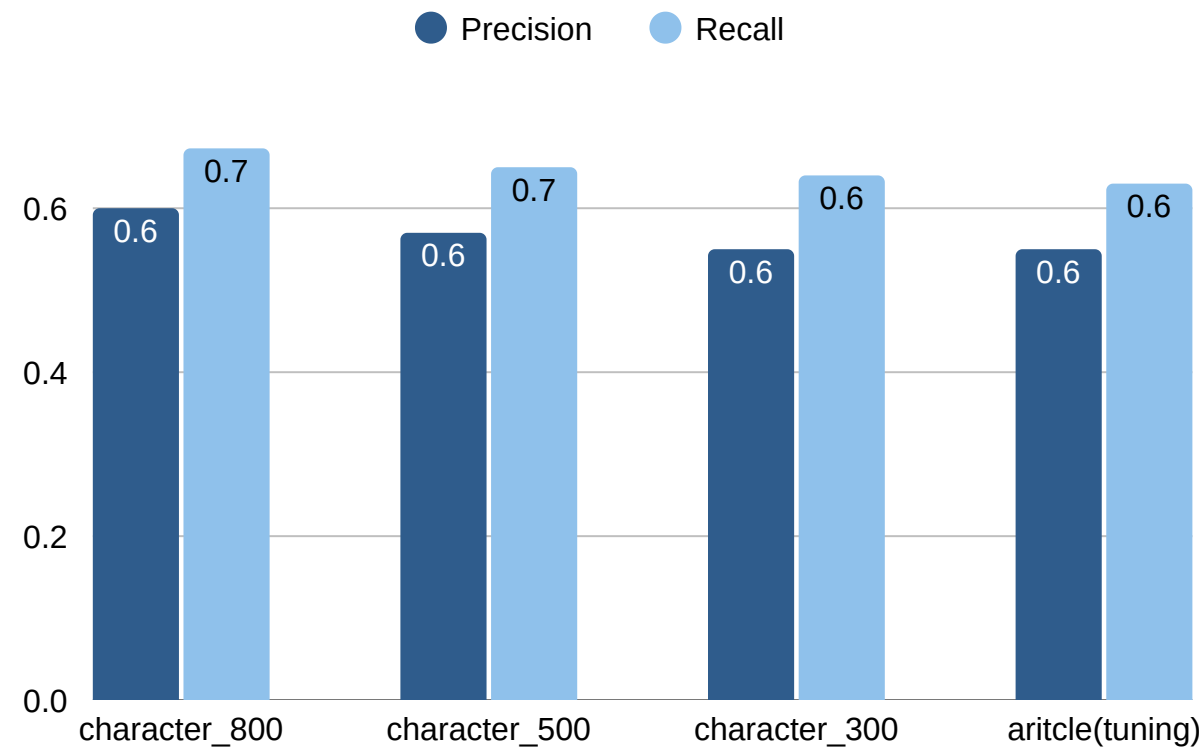
기존 baseline 설정은 이번 문서에 맞춰 튜닝되지 않은 기본값 — 조문 대비 다소 애매한 크기라 일부는 잘리고 일부는 불필요한 인접 조문까지 포함되어 정확도·속도 모두 가장 낮은 결과가 나옴.

article (조문 단위) *F1 0.621* · 청크 수 563개

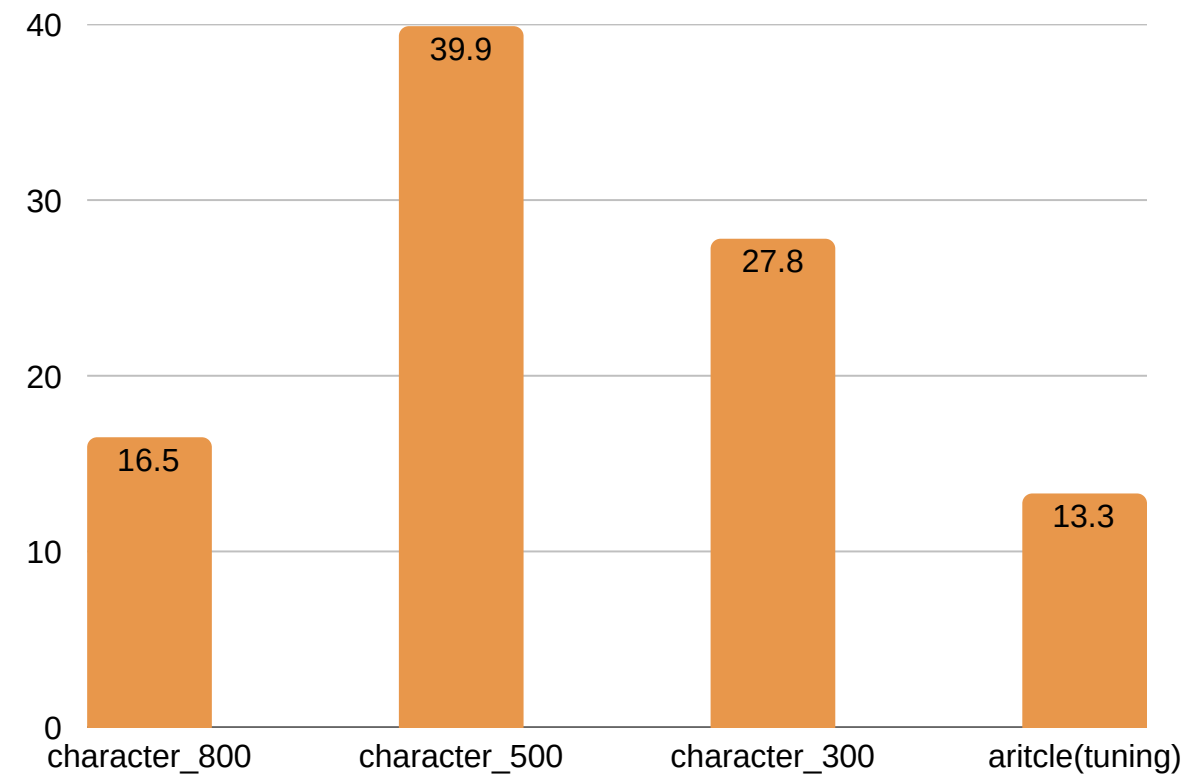
구조를 지켜 조문 단위로는 잘 나누지만, 짧은 조문은 전후 맥락(다른 조문 참조)이 빠지고 긴 조문은 800자 초과분만 별도 분할되어 청크 길이가 들쭉날쭉함 — 그 결과 baseline보다는 낫지만 character_800의 안정적인 맥락 양을 넘지 못함.

I 결과 ② 응답시간 / 청크 수 / precision / Recall

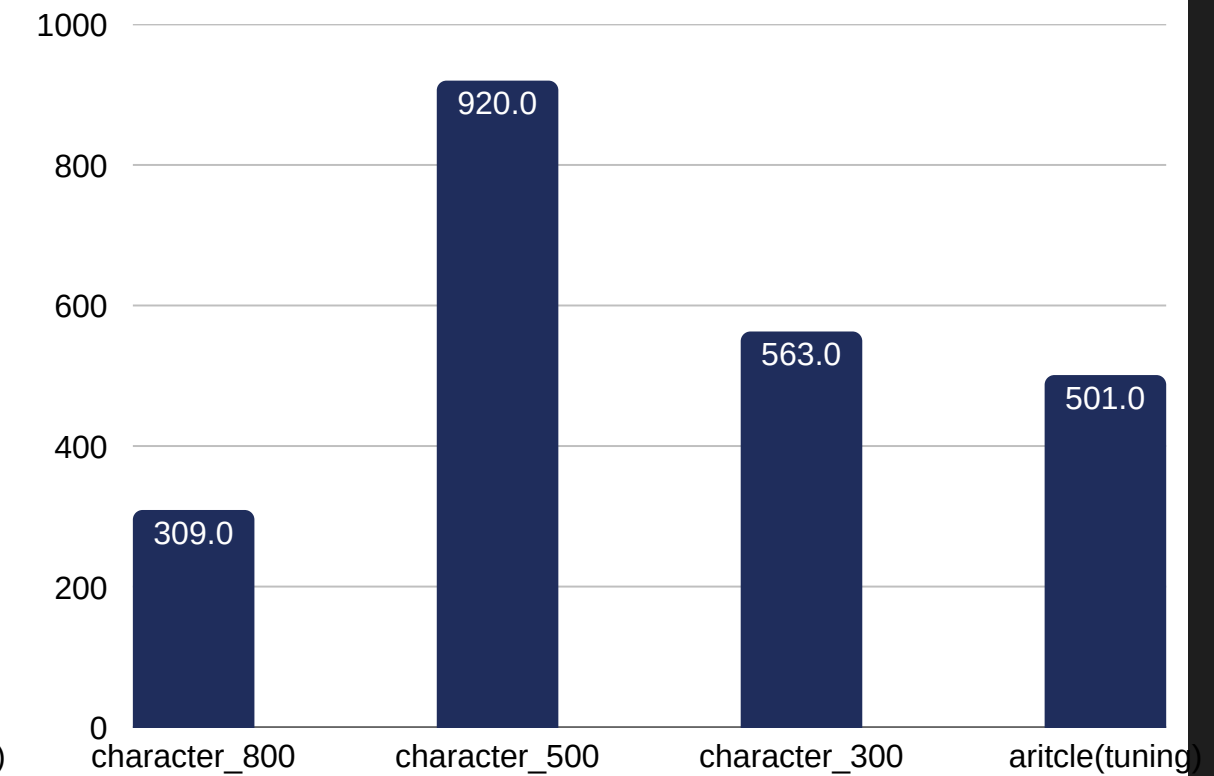
청킹 전략별 BERTScore Precision / Recall



청킹 전략별 평균 응답시간 (초)



청크 수 (개)



왜 이런 결과가 나왔을까?

청크 수는 적는데 Precision/Recall이 가장 높은 이유 (character_800) 청크 크기가 클수록(800자) 조문 하나가 청크 경계에서 잘리지 않고 온전히 한 청크에 담길 확률이 높아짐 → 검색된 청크 자체가 답변에 필요한 근거를 더 완전하게 포함 → Precision·Recall 모두 상승. 반대로 작은 청크(300자)는 청크 수는 많아도 조문이 여러 조각으로 쪼개져, 정답 근거가 일부 청크에만 흩어져 있으면 놓치기 쉬움

응답시간은 청크 수·크기와 뚜렷한 상관관계가 없는 이유 Retriever는 항상 상위 k=3개 청크만 가져오므로, 전체 청크가 300개든 900개든 검색 자체의 소요 시간 차이는 미미함. 응답시간의 큰 차이(13.3s ~ 39.9s)는 청킹 전략보다 LLM 생성 단계의 변동성(답변 길이, 로컬 GPU 상태)에서 비롯된 것으로 추정 — 청킹 전략만으로 응답시간을 단정하기는 어려움

I 그래서 무엇을 했는가

01

청킹 설정 확정

F1 최고 성능을 보인 character_800을 표준 청킹 전략으로 채택하고, config.py에 `chunk_size=800·chunk_overlap=100`을 최종 반영

02

벡터스토어 재구축

확정된 청킹 설정으로 Chroma 벡터스토어를 재생성하고, 동일 조건(top-k=3)의 Retriever·챗봇 파이프라인에 연결

03

회귀 검증

기존 평가 질의 10문항으로 재검증 — BERTScore F1 0.642, 응답시간 12.3초로 성능 재현 확인

04

한계 기록

article(조문 단위) 전략의 가변 분할 이슈(짧은 조문의 맥락 손실)를 다음 튜닝 과제로 기록 — 조문 간 참조가 많은 문서에 한해 추후 재검토 예정

I 인사이트

01

큰 청크(800)자가 법 조문 검색에는 더 유리했다.

청크가 작을수록(300자) 문맥이 조문 중간에서 잘려 의미가 끊기는 반면, 800자 청크는 조문 전후 맥락을 온전히 담아 검색 및 답변 정확도가 가장 높았습니다.

02

조문 단위 분할은 baseline보다 낮지만 최고는 아니었다.

제 0조 경계를 지키는 구조 인식형 분할이 baseline보다 F1이 높아, 구조를 아는 것이 도움은 되지만 이번 문서에서는 충분히 큰 문자 청크만 못했습니다.

03

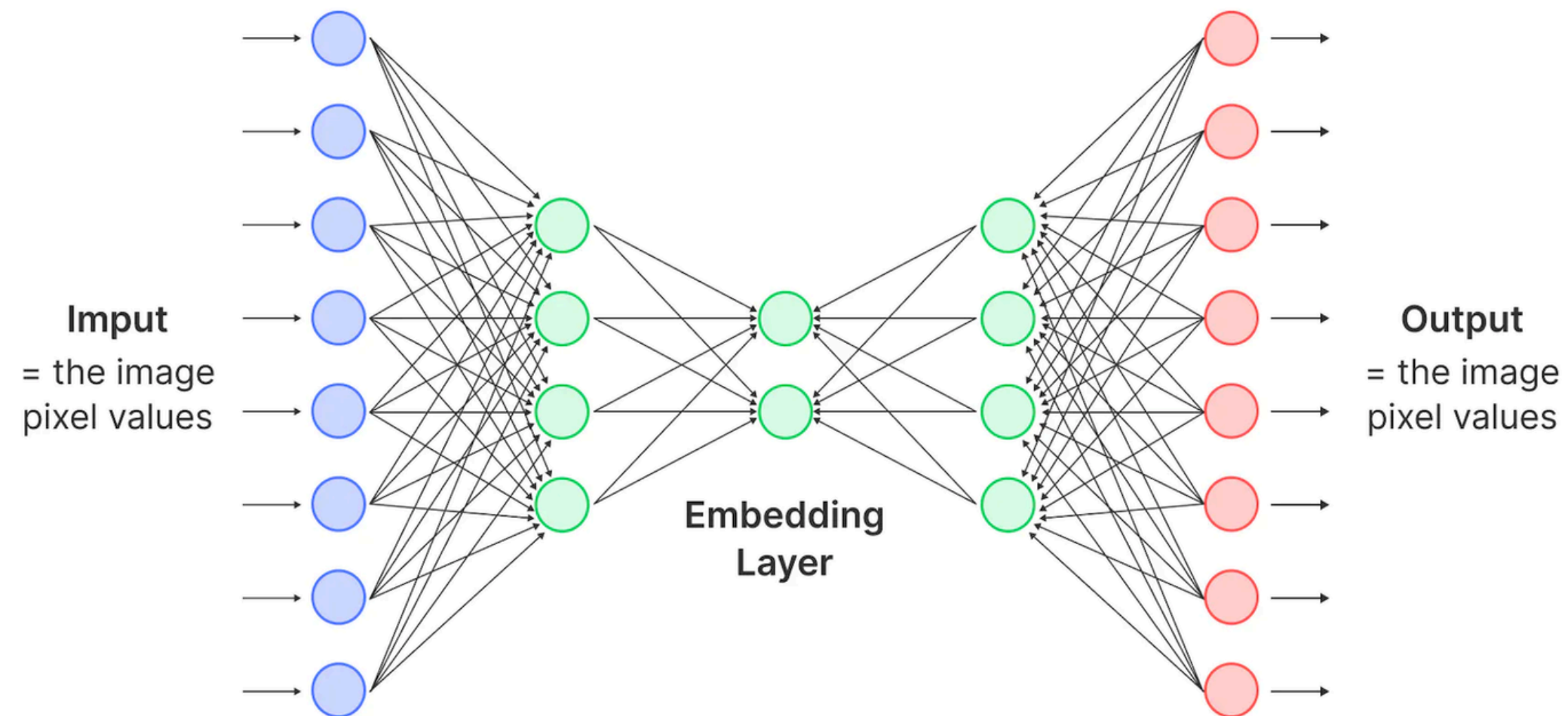
청크 수와 응답시간은 비례하지 않았다

청크가 가장 많은 character_300(920개)이 가장 빨랐고 청크가 가장 적은 baseline(501개)이 가장 느렸습니다. 응답시간은 검색 단계보다 LLM 생성 길이에 더 좌우된 것으로 보입니다.

I Embedding

목적

법령 문서에 대한 검색 정확도를 높이기 위해, 한국어 문맥에 최적화된 임베딩 모델을 선정하여 Retrieval 정확도와 RAG 응답 품질을 향상시키는 것



전체 파이프라인 속 위치

청킹

임베딩

검색

LLM

챗봇

I 실험 설계

공통 고정 조건

LLM
Qwen2.5-7B-Instruct (4bit, 로컬)

청킹
마크다운 기반, 500자 / overlap 50

검색 방식
similarity, top-k = 3

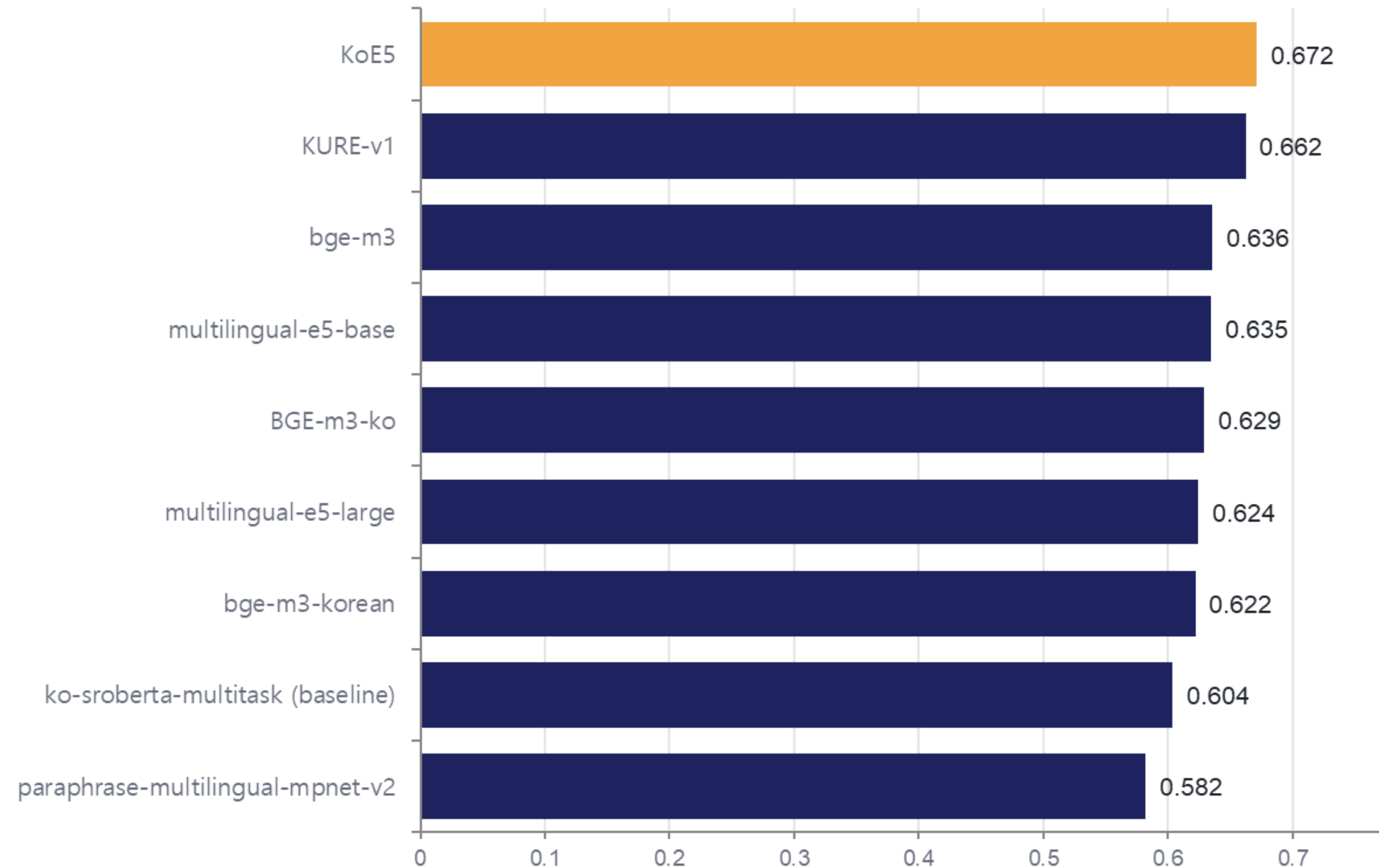
평가 데이터
자동차관리법 질의 10문항

평가 지표
BERTScore F1 등

비교 대상 임베딩 모델 (9종)

모델	유형
KoE5	한국어 특화
KURE-v1	한국어 특화
bge-m3	다국어
multilingual-e5-base	다국어
BGE-m3-ko	한국어 특화
multilingual-e5-large	다국어
bge-m3-korean	한국어 특화
ko-sroberta-multitask (baseline)	다국어
paraphrase-multilingual-mpnet-v2	다국어

I 결과 ① 정확도 (BERTScore F1)



최고 성능

0.671

KoE5(nlpai-lab)

baseline(0.604) 대비 + 11.2p%

Top 3

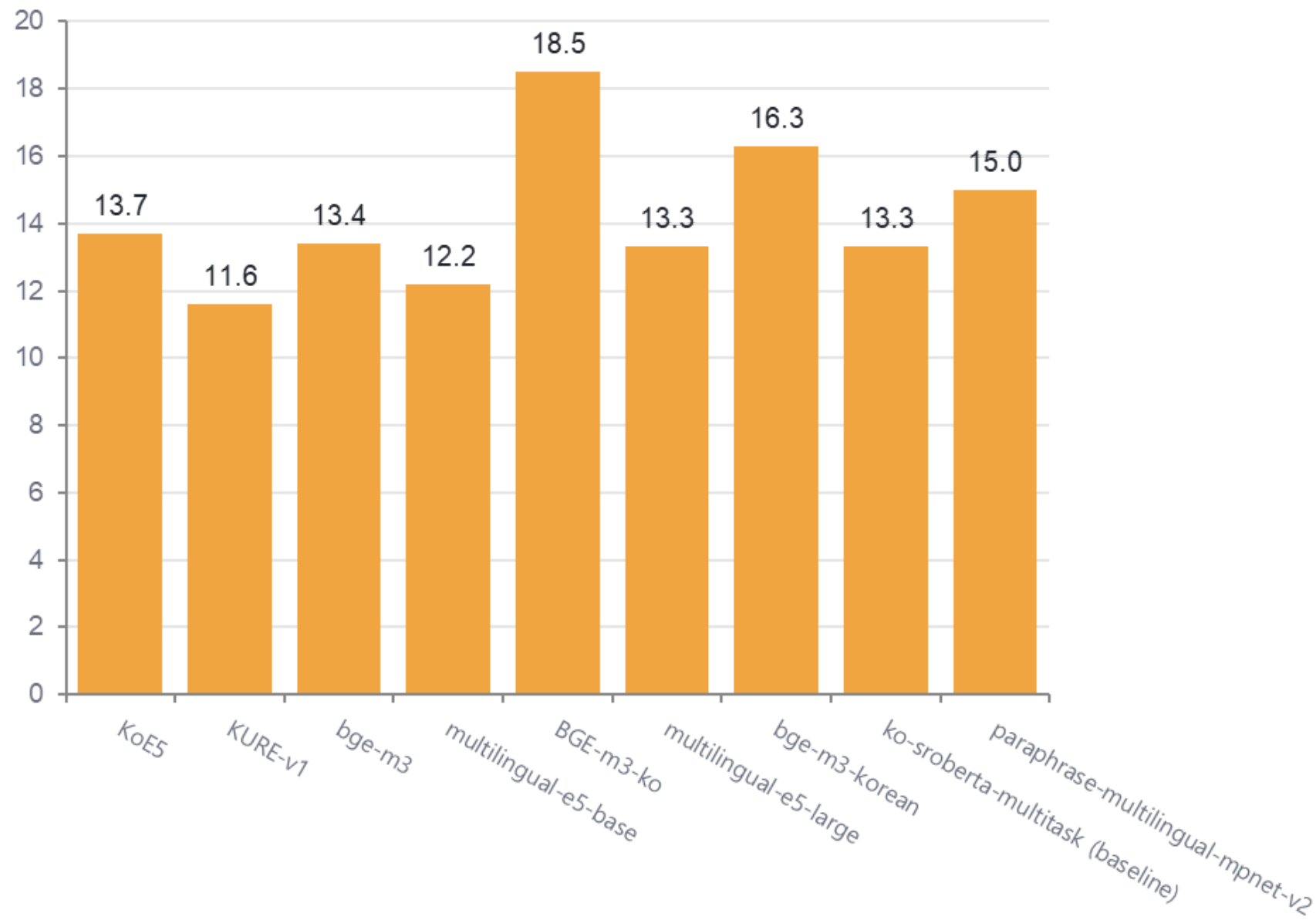
1. KoE5

2. KURE-v1

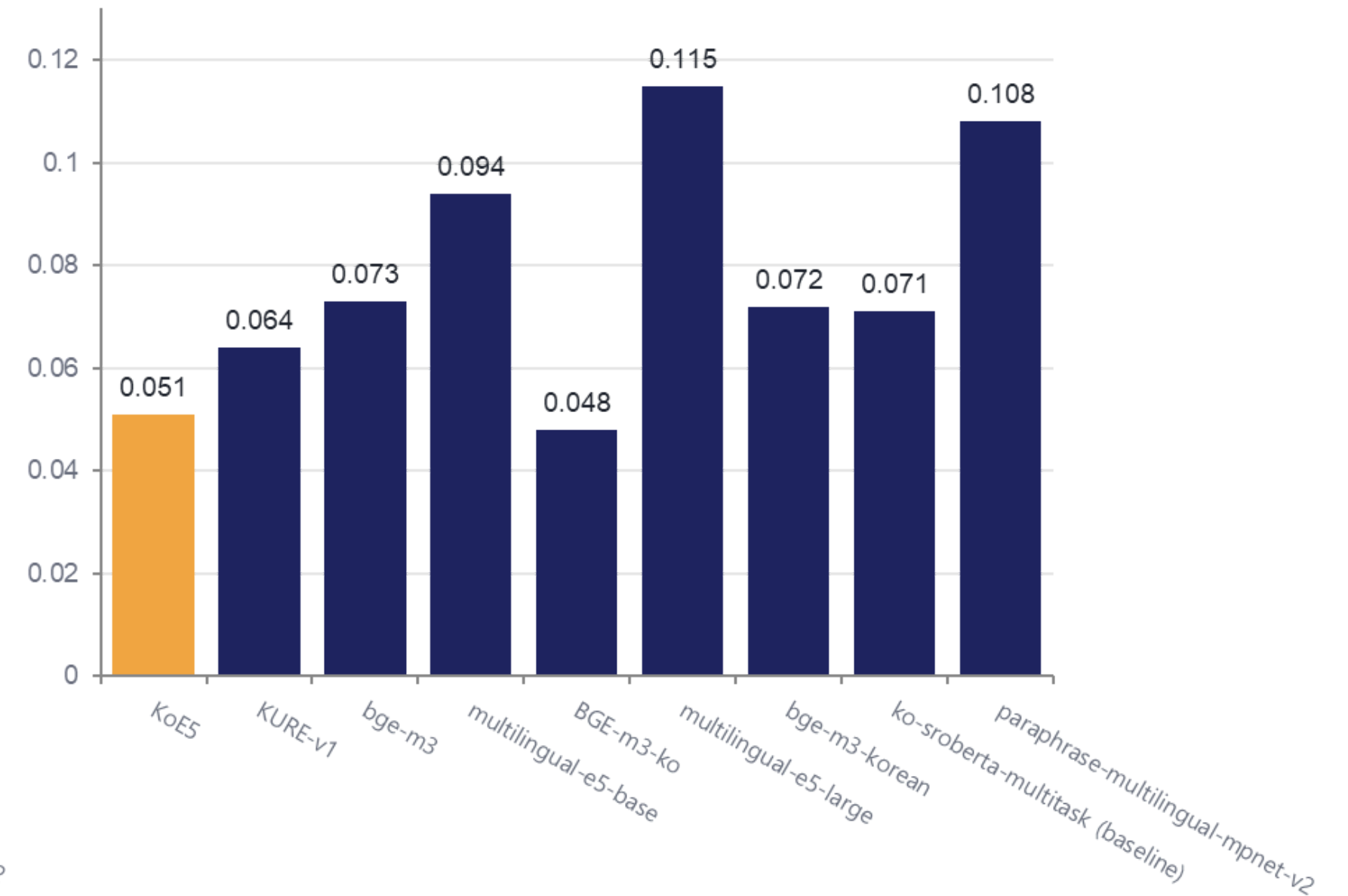
3. bge-m3

I 결과 ② 응답시간 · 답변 안정성

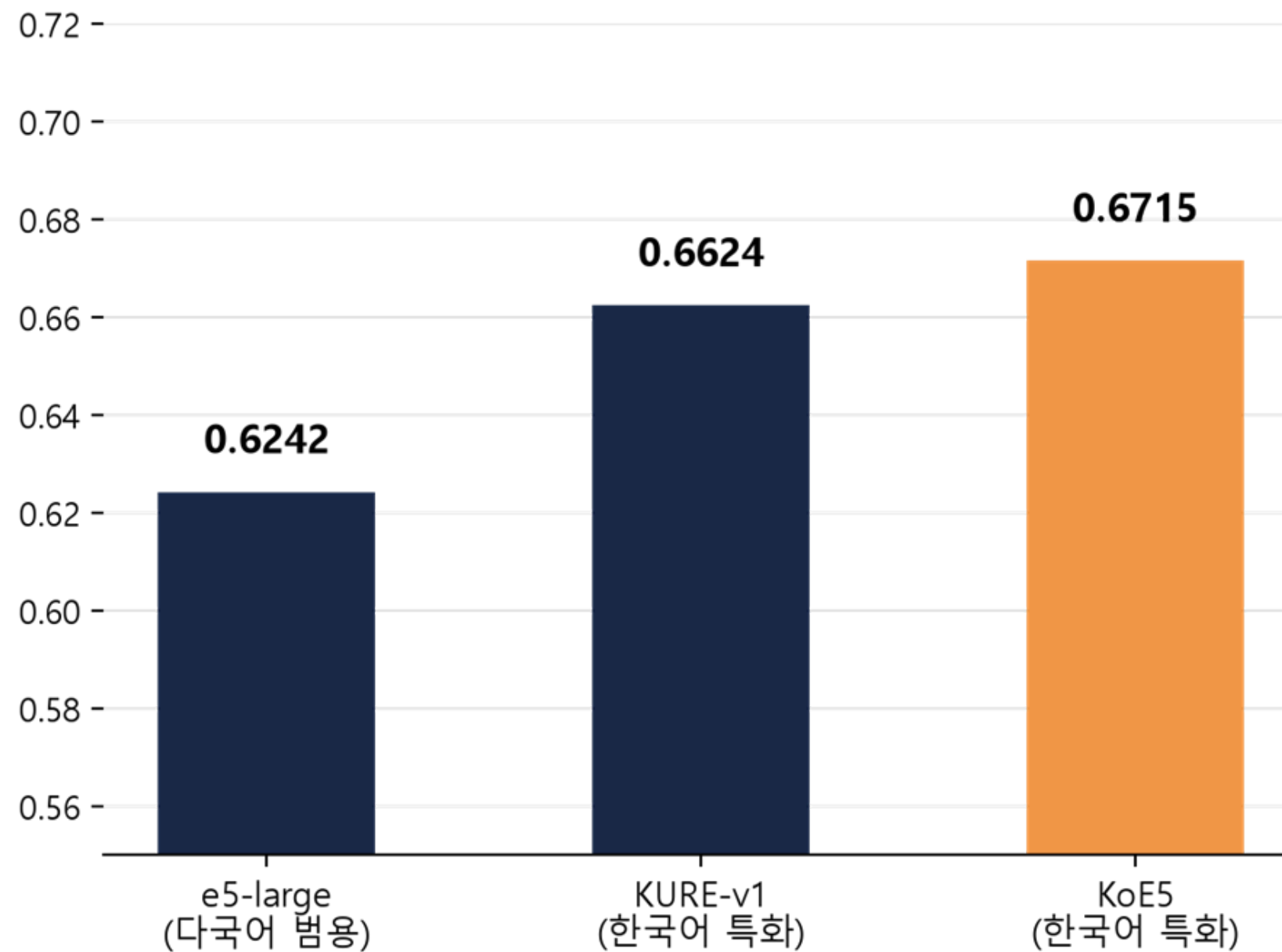
평균 응답시간 (초)



질문별 F1 표준편차 (낮을수록 안정적)



I 왜 KoE5인가 — 다른모델과의결정적차이



같은 구조,
다른 결과

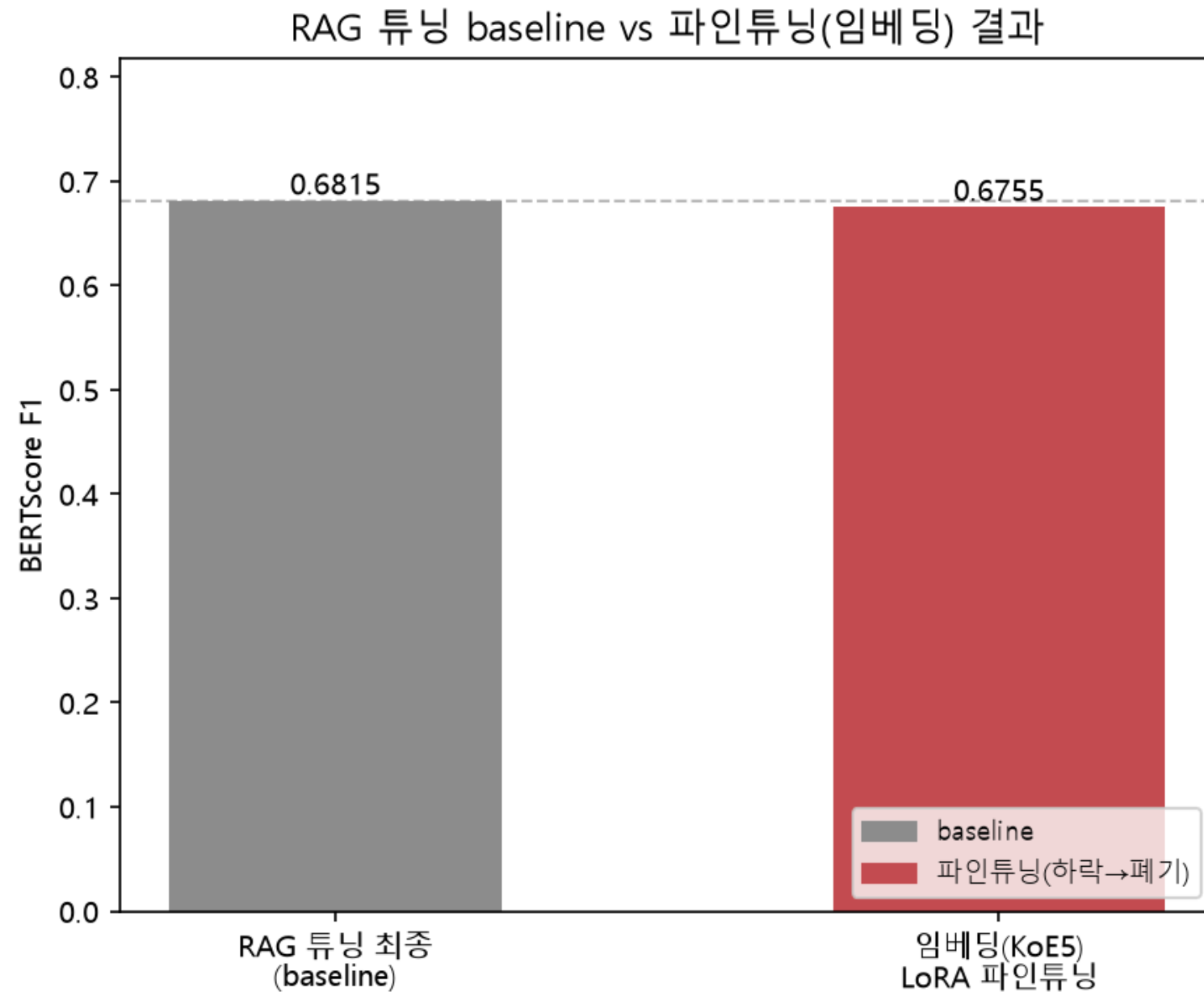
셋 다 e5-large 계열 — 구조는 동일

e5-large는 언어의 범용 데이터로 학습 → 한국어 하나에 집중하지 못함

즉, "한국어 검색 데이터로 재학습했는가" 만으로 차이 발생

"모델을 더 크게 만드는 것보다, 한국어 데이터로 다시 학습시키는 것이 더 효과적이었다."

I 파인튜닝 — 모델을 더 개선할 수 있을까



파인튜닝 전 (원본 KoE5)

0.6815

파인튜닝 후 (LoRA 대조학습)

0.6755

판정

파인튜닝 기각 (-0.0060)

"모델을 더 크게 만드는 것보다, 한국어 데이터로 다시 학습시키는 것이 더 효과적이었다."

I 1차 비교 vs 2차 비교 — 청킹방식

1차 비교

파싱 PyPDFLoader — 페이지 단위 원문 그대로 추출

분할 RecursiveCharacterTextSplitter — 구조 인식 없이 500자/overlap 50 단순 분할

문제 조문이 페이지·글자수 경계에서 끊겨 불완전한 청크 다수 발생

1위 **multilingual-e5-base**

2차 비교

파싱 pymupdf4llm — PDF를 마크다운 구조(조문 헤더 등)로 변환

분할 RecursiveCharacterTextSplitter.from_language(MARKDOWN) — 구조를 인식하며 분할

개선 청크가 조문 경계에 맞춰져 문맥이 온전한 후보군 확보

1위 **KoE5 (F1 0.671)**

결론 "어떤 임베딩이 좋은가"를 묻기 전에 "문서를 어떻게 자르느냐"부터 고정해야 공정한 비교가 성립한다 — 이후 모든 비교는 마크다운 기준(2차)으로 통일해서 진행

1차 비교 VS 2차 비교 — 청킹방식

1차 비교 (Baseline)

파싱 PyPDFLoader — 페이지 단위 물리적 텍스트 추출

분할 RecursiveCharacterSplitter — 구조 인식 없는 500자 단순 분할

문제 조문이 페이지 및 글자 수 경계에서 단절되어 맥락 소실 및 불완전한 청크 다수 발생

1위 multilingual-e5-base

2차 비교 (Optimized)

파싱 pymupdf4llm — PDF를 구조적인 마크다운(Markdown)으로 전처리

분할 from_language(MARKDOWN) — 문서 계층 구조를 인식한 분할

개선 청크가 조문 경계에 정확히 일치하여 온전한 법률 맥락 보존 및 검색 품질 향상

1위 KoE5 (F1 0.671)

I 인사이트

01

한국어 특화 임베딩이 다국어 범용 모델을 앞섰다

koE5와 KURE-v1이 나란히 1,2위를 차지 다국어 대형 모델 보다
한국어 파인튜닝 모델이 법 조문 검색에서 더 정확

02

정확도가 응답 안정성과 함께 움직였다

koE5 및 BGE-m3-ko는 질문마다 답변 품질이 고르게 유지된 반면 multilingual-e5-large 및 paraphrase-mpnet은
편차가 커 특정 질문에서 크게 혼동

03

소규모 데이터 파인튜닝은 역효과가 날수 있다.

387개 합성 QA로 KoE5 파인튜닝했지만 F1이 오히려 하락(0.6815-> 0.6755)
파인튜닝은 항상 개선이라는 가정은 실측 없이는 믿을수 없음.

I 검색(Retrieval) 개요

목적

질문과 의미가 비슷한 문서뿐 아니라, 조항 번호나 용어처럼 정확히 일치해야 하는 키워드가 담긴 문서도 함께 찾을 수 있도록 BM25와 Dense를 결합해서, 검색 정확도와 챗봇 답변 품질을 높이는 것

왜 하이브리드 검색인가?

- Dense 단독은 의미는 비슷해도 조항 번호나 수치가 다른 문서를 헛갈릴 수 있음
- BM25는 정확한 용어는 잘 찾지만 문맥적인 유사성은 못 잡음
- 두 방식을 함께 쓰면 서로의 약점을 보완해서 검색 정확도가 올라감

전체 파이프라인 속 위치

청킹

임베딩

검색

담당

임베딩

챗봇

I 실험 설계

공통 고정 조건

LLM

Qwen2.5-7B-Instruct (4bit, 로컬)

임베딩 모델

jhgan/ko-sroberta-multitask

검색 방식

similarity, top-k = 3

평가 데이터

자동차관리법 질의 10문항

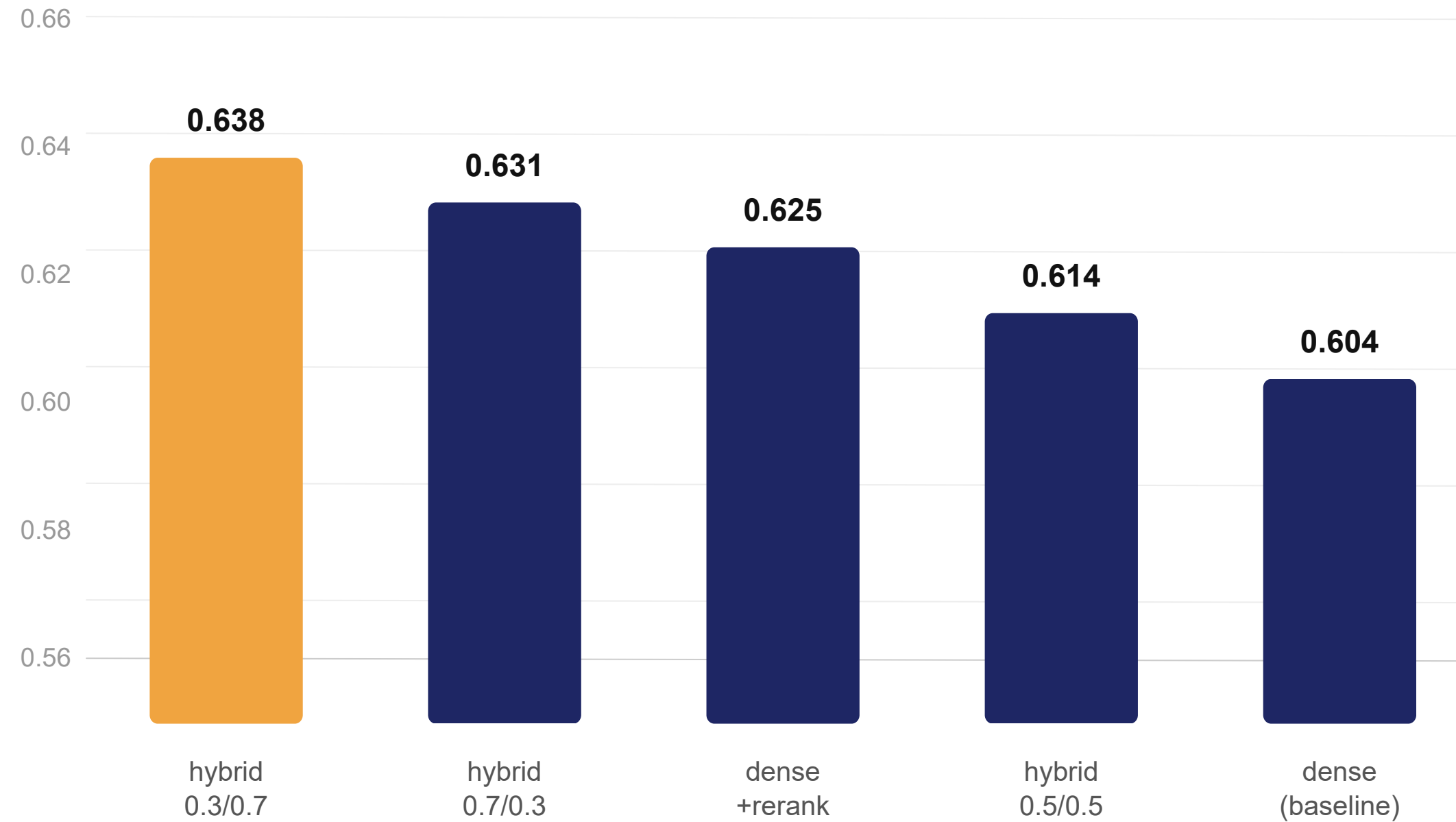
평가 지표

BERTScore F1 등 (뒤에서 확장)

비교 대상 검색 전략

전략	구성
Dense (baseline)	임베딩 유사도 검색만 사용 (k=3)
Hybrid 0.5 / 0.5	BM25 0.5 + Dense 0.5 앙상블
Hybrid 0.3 / 0.7	BM25 0.3 + Dense 0.7 앙상블
Hybrid 0.7 / 0.3	BM25 0.7 + Dense 0.3 앙상블
Dense + Rerank	Dense top-10 → Cross-Encoder(bge-reranker-v2-m3) top-3 재정렬

I 결과 ① 정확도 (BERTScore F1)

**채택****hybrid_0.3_0.7**

F1 0.6385 (baseline 대비 +5.7%)

BM25 0.3 + Dense 0.7 앙상블

왜 이 전략인가 BM25로 정확한 키워드를, Dense로 문맥이 비슷한 문서를 함께 찾아 서로의 약점을 보완했습니다. 그 결과 baseline(Dense 단독)보다 5.7% 높은 정확도를 기록했습니다.

I 결과② 그런데, 지표를 늘려보니 예상보다 낮았다

전략	BERTScore F1	응답시간(s)	Fact Match	ROUGE-L	MRR
hybrid_0.3_0.7	0.6385	23.6	0.10	0.204	0.483
hybrid_0.7_0.3	0.6315	25.1	0.00	0.290	0.320
dense_rerank	0.6245	17.4	0.00	0.158	0.600
hybrid_0.5_0.5	0.6142	22.9	0.20	0.228	0.358
dense (baseline)	0.6039	12.6	0.00	0.120	0.433

실제 답변 예시 (dense)

Q. 비사업용 승용자동차의 정기검사 유효기간과, 기간을 경과했을 때 부과되는 과태료 기준은 무엇인가요?

A. "정기검사 유효기간과 과태료 기준에 대한 정보는 주어진 문맥에서 직접적으로 찾을 수 없습니다. 따라서, 문맥에 없는 내용임을 알려드립니다."

문제 위 예시처럼, 검색이 필요한 문서를 못 찾으면 모델은 지어내지 않고 정직하게 모른다고 답합니다. 정확하지만 답을 안 한 것이니 Fact Match는 0점 처리됩니다. 왜 이런 경우가 생기는지 더 파고들었습니다.

I 1차 개선 시도: 튜닝으로 해결될까

시도 1

BM25 단어 인식 방식 바꾸기

한국어는 조사가 붙으면 같은 단어도 다르게 인식됩니다. '제34조'와 '제34조에'를 서로 다른 단어로 보는 식이에요. 그래서 조항 번호는 항상 하나의 단어로 따로 인식하도록 방식을 바꿔서 적용해봤습니다.

결과: 검색 순위에 변화가 없었습니다. 저희가 쓰는 결합 방식 (Ensemble)은 점수가 아니라 순위를 기준으로 합치기 때문에, 단어 인식이 좋아져도 순위 자체가 안 바뀌면 결과에 반영되지 않는다는 걸 확인했습니다.

시도 2

검색 개수(k) 늘려보기

검색 결과를 더 많이 가져오면 필요한 정보를 놓치지 않을 수도 있다고 생각했습니다. 그래서 기존에 3개씩 가져오던 검색 결과를 5개, 10개로 늘려가며 성능을 비교했습니다.

결과: 5개일 때 가장 좋았고(0.57점), 오히려 10개로 늘렸을 때 더 나빠졌습니다(0.44점). 검색 후보가 너무 많아지면 순위가 다시 섞이면서 방해가 될 수 있다는 걸 알게 됐습니다.

그래서 두 방법 모두 부분적인 도움은 됐지만, 뭔가 더 근본적인 원인이 있다고 판단해서 다음 단계로 넘어갔습니다.

I 2차 원인 분석: 질문이 너무 복잡했던 걸까

가장 어려운 질문 하나는 두 조항(제43조, 제84조)을 동시에 알아야 답할 수 있었습니다. 그런데 원문에서 이 두 조항이 9만 자 넘게 떨어져 있다는 걸 발견했습니다. 너무 멀리 떨어져 있어서 한 번의 검색으로 둘 다 찾기 어려운 게 아닐까 의심했습니다.

그래서 실제로 질문을 2개로 나눠서 다시 검색해봤습니다

Q 비사업용 승용자동차의 최초 등록 후 정기검사 유효기간은 어떻게 되나요?



제43조 문서를 2위로 찾음

질문을 단순하게 나누니 검색이 더 쉬워졌습니다

→ 이 조항은 검색이 어렵지 않았음.

Q 자동차 정기검사 기간을 경과했을 때 부과되는 과태료 기준은 어떻게 되나요?



제84조 문서를 끝내 찾지 못함

질문을 단순하게 나눠도 결과는 똑같았습니다

→ 질문을 아무리 단순하게 만들어도 안 잡히는 걸 보면, 질문이 복잡해서가 아니라 다른 이유가 있는 것으로 예상

I 발견: 평가 데이터(정답) 자체의 오류

질문	기존 (오류)	원문 확인 결과
정기검사·과태료	"30만원", 원문에 없음	제84조제4항: 100만원 이하
성능·상태 점검기록부	"제58조", 그런 조항 없음	제58조의4가 정확한 조항
말소등록 기한	"1개월"은 있지만 뜻이 다름	관청 직권말소 사전통지 기한이지 소유자 신청기한 아님
자율주행 임시운행허가	"제27조의2", 존재하지 않음	제27조만 존재
이전등록 과태료	"50만원", 원문 어디에도 없음	검증 불가, 근거 없음

왜 중요한가 질문과 정답이 담긴 이 파일은 A, B, C, D 4개 파트가 함께 쓰는 파일입니다. 검색 파트만의 문제가 아니라 팀 전체 평가에 영향을 준 문제였고, GitHub 이슈로 공유하고 팀 공통 파일을 수정했습니다.

I 재채점 결과

전략	BERTScore F1	Fact Match	ROUGE-L
hybrid_0.3_0.7	0.6385 → 0.6552	0.10 → 0.30	0.2042 → 0.2148
hybrid_0.7_0.3	0.6315 → 0.6472	0.00 → 0.10	0.2905 → 0.2216
dense_rerank	0.6245 → 0.6403	0.00 → 0.10	0.1575 → 0.1700
hybrid_0.5_0.5	0.6142 → 0.6330	0.20 → 0.30	0.2284 → 0.1947
dense (baseline)	0.6039 → 0.6183	0.00 → 0.20	0.1201 → 0.0909

- ✓ BERTScore는 5개 전략 전부 올랐습니다. 정답이 틀려서 억울하게 감점되던 부분이 회복된 것입니다.
- ✓ hybrid_0.3_0.7이 더 확실한 1위(0.6552)가 되어, 처음 채택했던 결론이 다시 한번 확인됐습니다.
- ✓ ROUGE-L은 일부 내려갔습니다. 정답 문구를 정직하고 짧게 고치다 보니 생긴 자연스러운 결과입니다.

I 인사이트

01

성능 문제의 원인이 항상 알고리즘에 있는 건 아니다

단어 인식 방식을 바꾸고, 검색 개수를 조정하고, 질문을 나눠보기까지 했지만 전부 큰 효과가 없었습니다. 진짜 원인은 검색이 아니라 평가 데이터 자체의 오류였습니다.

02

순위로 합치는 방식은 점수가 좋아져도 순서가 그대로면 안 바뀐다

단어 매칭이 좋아져도 최종 순위가 안 바뀌면 결과에 반영되지 않고, 후보를 늘리면 오히려 순위가 다시 섞여 성능이 떨어질 수 있다는 걸 실험으로 확인했습니다.

03

정답 데이터도 검증 대상이다

원문과 직접 대조하지 않았다면 몰랐을 오류를 찾아 고쳤고, 다시 채점해보니 모든 지표가 좋아지는 걸 숫자로 확인했습니다.

I 한계점

01

점수를 크게 끌어올릴 정확한 원인은 다 찾지 못했습니다

정답 데이터를 고친 뒤에도 Fact Match는 가장 높은 값이 0.30입니다. 검색된 내용과 실제 답변을 하나하나 비교하는 절차까지 갔다면 더 정확한 원인을 찾았을 것 같은데, 시간 관계상 거기까지는 못 갔습니다.

02

검색은 잘해도 답변이 못 따라가는 경우가 있었습니다

dense_rerank 전략은 검색 정확도(MRR)가 제일 높았지만 Fact Match는 0.10에 그쳤습니다. 프롬프트에 검색된 내용을 더 적극적으로 반영 하라는 지시를 추가했다면 결과가 달랐을 수도 있습니다.

03

리랭커 파인튜닝은 아직 진행 중입니다

학습에 11시간 가까이 걸려서 이번 발표에는 결과를 반영하지 못했습니다. 더 일찍 시작했다면 오늘 결과에 포함할 수 있었을 것 같습니다.

I LLM 모델 비교 개요

목적

RAG 파이프라인의 최종 답변 생성 품질을 극대화하기 위해, 로컬 GPU(8GB)에서 구동 가능한 다양한 LLM 모델을 비교해 법령 QA에 가장 적합한 모델을 선정하는 것

왜 LLM 모델을 비교하였는가?

- LLM은 검색된 문맥을 실제 답변으로 바꾸는 마지막 단계라, 어떤 모델을 쓰느냐가 최종 답변 품질에 가장 직접적인 영향을 줄 것으로 예상됨
- baseline(Qwen2.5-7B)은 편의상 고정한 값일 뿐, 이 도메인(법령 QA)에 가장 적합한 모델 인지는 아직 검증된 바 없음
- 모델 파라미터 수가 클수록 성능이 좋다는 보장은 없으므로, 소형·특화 모델까지 폭넓게 실측 비교해볼 필요가 있음

전체 파이프라인 속 위치

청킹

임베딩

검색

LLM

챗봇

I 실험 설계

비교 대상 LLM 모델

공통 고정 조건

임베딩
jhgan/ko-sroberta-multitask

청킹
마크다운 기반, 500자 / overlap 50

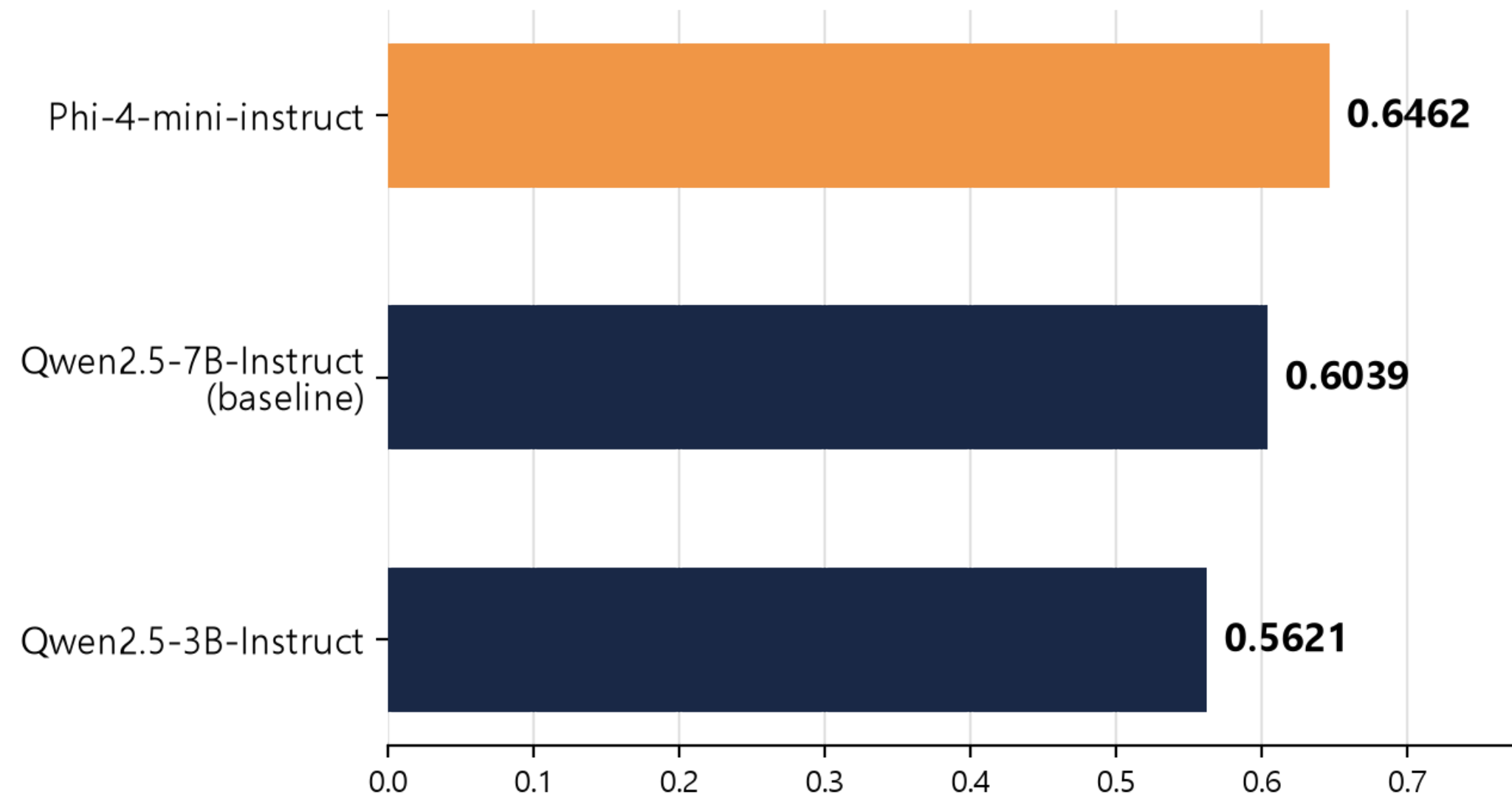
검색 방식
similarity, top-k = 3

평가 데이터
자동차관리법 질의 10문항

평가 지표
BERTScore F1 등

모델	유형	F1
Phi-4-mini-instruct	Microsoft, 3.8B	0.6462
Qwen2.5-7B-Instruct (baseline)	Alibaba, 7B	0.6039
Qwen2.5-3B-Instruct	Alibaba, 3B	0.5621

I 결과 ① 정확도 (BERTScore F1)



최고 성능

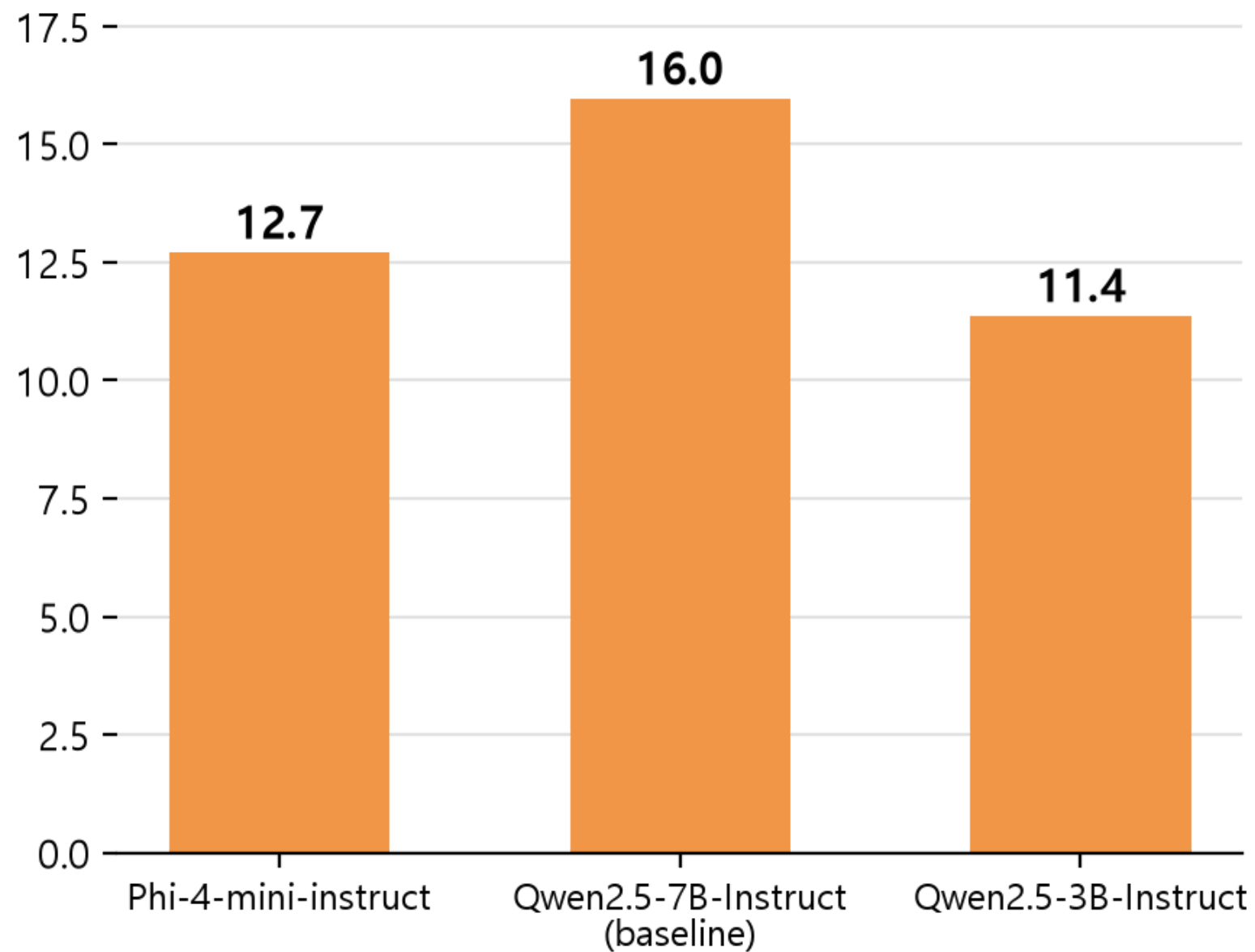
0.6462

1차: 7B가 3B를 유의미하게 앞섬

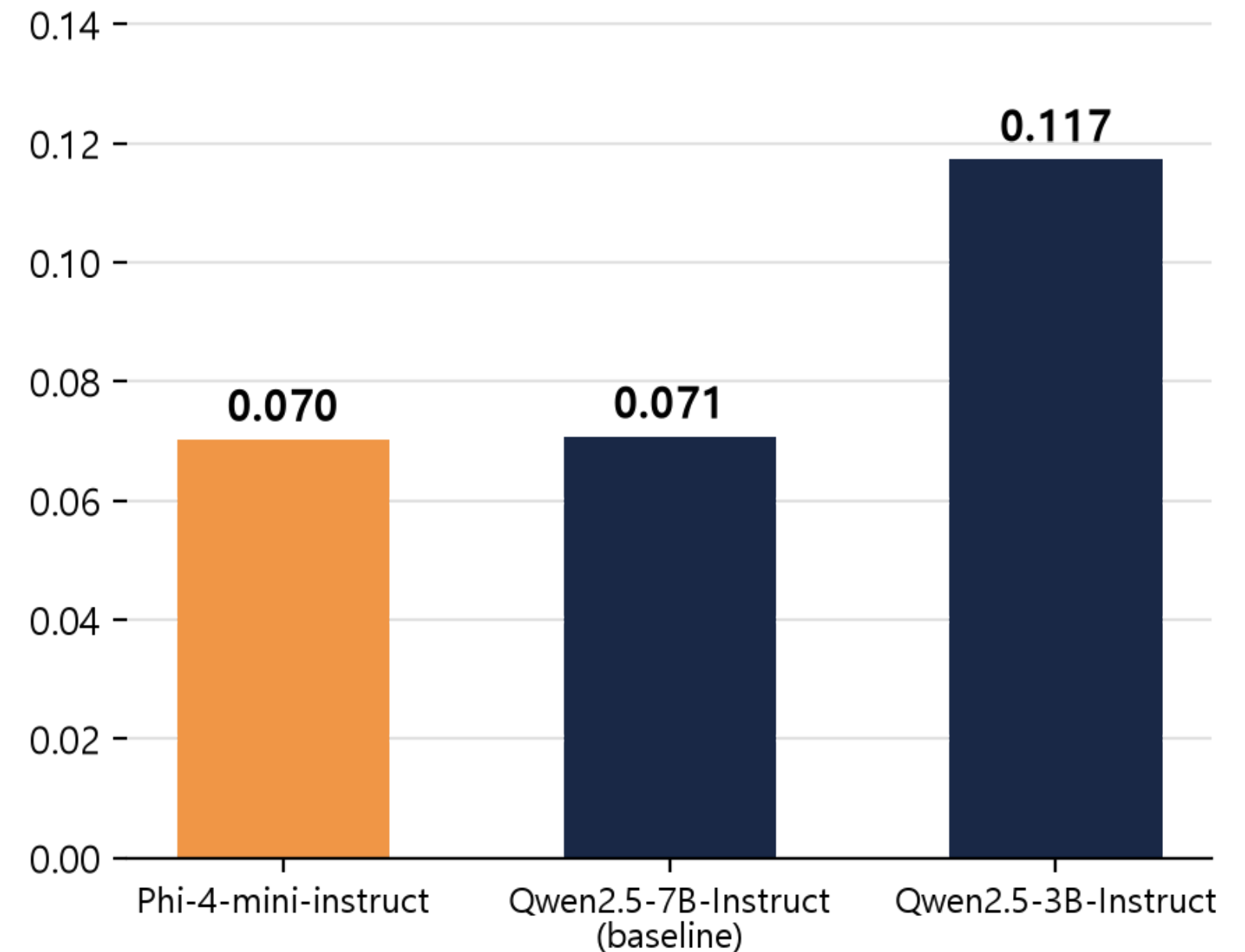
2차 확장에서 소형(3.8B) Phi-4-mini가
baseline 7B를 역전

I 결과 ② 응답시간 · 답변 안정성

평균 응답시간 (초)



질문별 F1 표준편차 (낮을수록 안정적)



! BERTScore만으로는 부족했다

BERTScore F1

0.6462

Fact Match

0.10

ROUGE-L

0.143

Fact Match·ROUGE-L이란?

Fact Match: 정답(ground_truth)에 나온 조항 번호(제○조)가 실제 답변에도 그대로 들어있는지 (0 또는 1)

ROUGE-L: 정답과 답변 사이에 같은 순서로 겹치는 어구가 얼마나 되는지 — 문구 자체의 유사도

→ BERTScore는 의미가 비슷하면 높게 나오지만, 조항 번호가 틀려도 의미상 비슷하면 점수가 크게 안 떨어짐

→ Fact Match 0.10 = 10문항 중 정확한 조항 번호를 맞힌 게 단 1개뿐이라는 뜻

I 카테고리별 강약점

질문 카테고리	Qwen2.5-3B	Qwen2.5-7B	Phi-4-mini
말소 등록 (폐차 등)	0.369	0.550	0.637
명의이전(이전등록)	0.516	0.532	0.661
불법 튜닝 및 처벌	0.428	0.460	0.505
이륜자동차(오토바이) 관리	0.650	0.626	0.590
자동차 등록 번호판	0.582	0.630	0.644
자율주행차 및 임시운행허가	0.706	0.700	0.750
정기검사 주기 및 과태료	0.450	0.640	0.690
제작결함 시정(리콜)	0.669	0.670	0.648
중고차 성능·상태 점검	0.575	0.613	0.607
튜닝 관련	0.678	0.621	0.729

I 최종 모델 선정

KoE5 + Phi-4-mini

0.6618

KoE5 + Qwen2.5-7B

0.6715

✓ LLM 측 비교에서 **Phi-4-mini**가 더 높게 나온 이유는 임베딩이 jhgan으로 고정된 조건에서였음

✓ 임베딩을 KoE5로 바꾼 뒤 같은 비교를 다시 하니, 오히려 **Qwen2.5-7B** 쪽이 더 높게 나옴

✓ 그래서 최종 파이프라인의 LLM은 baseline **Qwen2.5-7B**로 유지됨

I 인사이트

01

작은 모델이 큰 모델을 이길 수 있음

세 하이브리드 조합 중 BM25:Dense=0.5:0.5(F1 0.614)가 오히려 가장 낮았고, 0.3:0.7(0.639)과 0.7:0.3(0.632)이 더 높았다
단순 평균보다 한쪽에 무게를 둔 비대칭 결합이 이 데이터셋엔 더 잘 맞았다.

02

속도를 포기해야 정확도를 얻는 건 아니었음

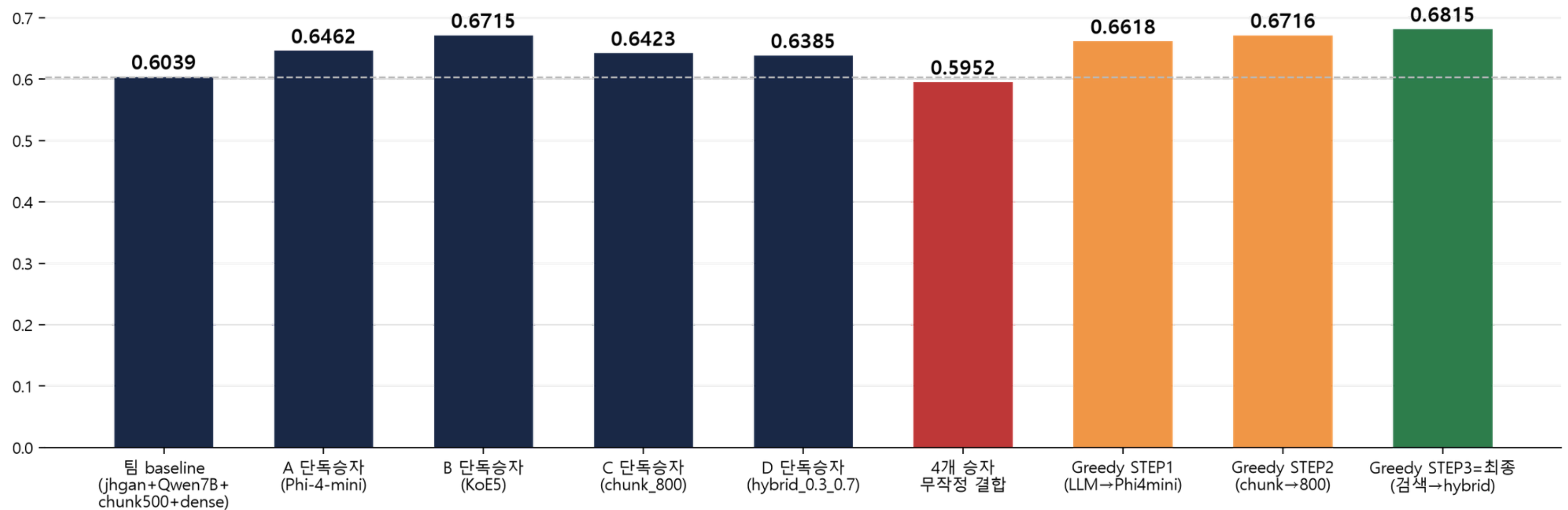
Phi-4-mini는 baseline보다 3.3초 더 빠르면서 F1도 더 높음
무조건 큰 모델을 쓸 이유가 없다는 것을 실측으로 확인

03

대형 모델은 로컬 8GB GPU의 한계를 넘기 쉬움

14B/8B급 3종이 로드·실행 단계에서 실패
try/except 방어 코드로 실패가 전체 실험을 중단시키지 않도록 설계해 나머지 결과를 안전하게 확보

I 종합 결과 - 1



예상: 각자 제일 좋은 걸 합치면 전체도 제일 좋을 것 → 실제: baseline(0.6039)보다도 낮음

I 종합 결과 - 2

BERTSCORE F1
0.6815
baseline 대비
+12.9%
(0.6039 → 0.6815)

✓ 각 축(A/B/C/D)을 독립적으로 비교했을 때의 높은 모델을 그대로 합치면 오히려 성능이 떨어짐(F1 0.5952) → 축 간 상호작용 효과 확인

✓ 이미 검증된 조합(KoE5+Qwen7B+chunk500+dense, F1 0.6715)을 출발점 삼아, 변수 하나씩만 바꾸는 greedy 탐색으로 재검증

✓ 그 결과 LLM은 Phi-4-mini가 아닌 baseline Qwen2.5-7B가 최종 채택됨

경청해 주셔서
감사합니다!